

Programación concurrente y Tiempo Real

Tercera Edición

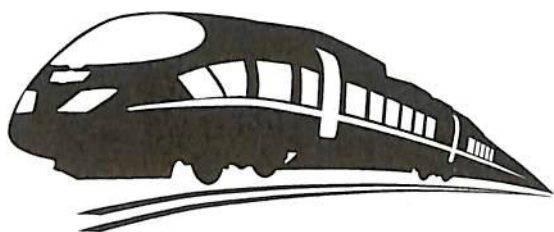
David Vallejo Fernández
Carlos González Morcillo
Javier A. Albusac Jiménez



005.133
JAV

Programación Concurrente y Tiempo Real

Tercera Edición



BIBLIOTECA
UNIVERSIDAD de PALERMO

David Vallejo Fernández
Carlos González Morcillo
Javier A. Albusac Jiménez

<SISTEMAS operativos>
<Lenguajes de programación>
<PROGRAMACIÓN de computadores>
<HARDWARE>
<Pascal> <C> <C++>

UNIVERSIDAD DE PALERMO BIBLIOTECA

056462

Procedencia: COMPRA

Fecha de ingreso: 24/11/2020



Título: Programación Concurrente y Tiempo Real
Edición: 3ª Edición - Enero 2016
Autores: David Vallejo Fernández, Carlos González Morcillo y Javier A. Albusac Jiménez
ISBN: 978-1518608261
Edita: David Vallejo Fernández

Printed by CreateSpace, an Amazon.com company
Available from Amazon.com and other online stores

Este libro fue compuesto con LaTeX a partir de una plantilla de Carlos González Morcillo y David Villa Alises.



Creative Commons License. Usted es libre de copiar, distribuir y comunicar públicamente la obra, bajo las condiciones siguientes: 1. Reconocimiento: Debe reconocer los créditos de la obra de la manera especificada por el autor o el licenciador. 2. No comercial: No puede utilizar esta obra para fines comerciales. 3. Sin obras derivadas: No se puede alterar, transformar o generar una obra derivada a partir de esta obra. Más información en: <http://creativecommons.org/licenses/by-nc-nd/3.0/>

3

Capítulo

Paso de Mensajes

En este capítulo se discute el concepto de **paso de mensajes** como mecanismo básico de sincronización entre procesos. Al contrario de lo que ocurre con los semáforos, el paso de mensajes ya maneja, de manera implícita, el intercambio de información entre procesos, sin tener que recurrir a otros elementos como por ejemplo el uso de segmentos de memoria compartida.

El paso de mensajes está orientado a los sistemas distribuidos y la sincronización se alcanza mediante el uso de dos primitivas básicas: i) el envío y ii) la recepción de mensajes. Así, es posible establecer esquemas basados en la recepción bloqueante de mensajes, de manera que un proceso se quede bloqueado hasta que reciba un mensaje determinado.

Además de discutir los conceptos fundamentales de este mecanismo de sincronización, en este capítulo se estudian las primitivas POSIX utilizadas para establecer un sistema de paso de mensajes mediante las denominadas **colas de mensajes POSIX** (*POSIX Message Queues*).

Finalmente, este capítulo plantea algunos problemas clásicos de sincronización en los que se utiliza el paso de mensajes para obtener una solución que gestione la ejecución concurrente de los mismos y el problema de la sección crítica, estudiado en el capítulo 1.

3.1. Conceptos básicos

Dentro del ámbito de la programación concurrente, el paso de mensajes representa un mecanismo básico de comunicación entre procesos basado, principalmente, en el envío y recepción de mensajes.

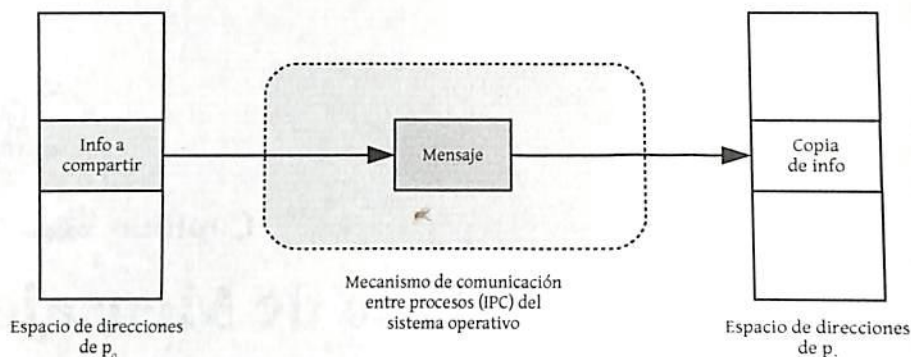


Figura 3.1: Uso de mensajes para compartir información entre procesos.

En realidad, el paso de mensajes representa una abstracción del concepto de comunicación que manejan las personas, representado por mecanismos concretos como por ejemplo el correo electrónico. La **comunicación entre procesos** (*InterProcess Communication*, IPC) se basa en esta filosofía. El proceso emisor genera un mensaje como un bloque de información que responde a un determinado formato. El sistema operativo copia el mensaje del espacio de direcciones del emisor en el espacio de direcciones del proceso receptor, tal y como se muestra en la figura 3.1.

En el contexto del paso de mensajes, es importante recordar que un emisor no puede copiar información, directamente, en el espacio de direcciones de otro proceso receptor. En realidad, esta posibilidad está reservada para el software en modo *supervisor* del sistema operativo.

En su lugar, el proceso emisor pide que el sistema operativo entregue un mensaje en el espacio de direcciones del proceso receptor utilizando, en algunas situaciones, el identificador único o PID del proceso receptor. Así, el sistema operativo hace uso de ciertas operaciones de copia para alcanzar este objetivo: i) recupera el mensaje del espacio de direcciones del emisor, ii) lo coloca en un buffer del sistema operativo y iii) copia el mensaje de dicho buffer en el espacio de direcciones del receptor.

A diferencia de lo que ocurre con los sistemas basados en el uso de semáforos y segmentos de memoria compartida, en el caso de los sistemas en paso de mensajes el sistema operativo facilita los mecanismos de comunicación y sincronización sin necesidad de utilizar objetos de memoria compartida. En otras palabras, el paso de mensajes permite que los procesos se envíen información y, al mismo tiempo, se sincronicen.

Desde un punto de vista general, los sistemas de paso de mensajes se basan en el uso de dos **primitivas de comunicación**:

- **Envío** (*send*), que permite el envío de información por parte de un proceso.
- **Recepción** (*receive*), que permite la recepción de información por parte de un proceso.

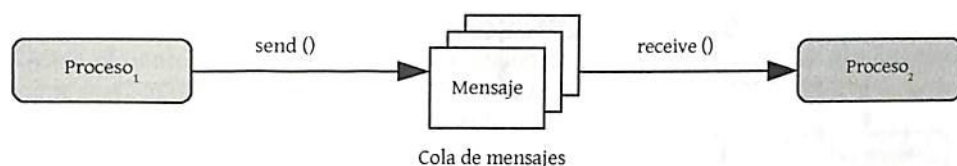


Figura 3.2: Esquema de paso de mensajes basado en el uso de colas o buzones de mensajes.

Típicamente, y como se discutirá en la sección 3.1.3, la **sincronización** se suele plantear en términos de llamadas bloqueantes, cuya principal implicación es el bloqueo de un proceso hasta que se satisfaga una cierta restricción (por ejemplo, la recepción de un mensaje).

3.1.1. El concepto de buzón o cola de mensajes

Teniendo en cuenta la figura 3.1, el lector podría pensar que el envío de un mensaje podría modificar el espacio de direcciones de un proceso receptor de manera espontánea sin que el propio receptor estuviera al corriente. Esta situación se puede evitar si no se realiza una copia en el espacio de direcciones del proceso receptor hasta que éste no la solicite de manera explícita (mediante una primitiva de recepción).

Ante esta problemática, el sistema operativo puede almacenar mensajes de entrada en una cola o buzón de mensajes que actúa como buffer previo a la copia en el espacio de direcciones del receptor. La figura 3.2 muestra de manera gráfica la comunicación existente entre dos procesos mediante el uso de un buzón o **cola de mensajes**.

3.1.2. Aspectos de diseño en sistemas de mensajes

En esta sección se discuten algunos aspectos de diseño [13] que marcan el funcionamiento básico de un sistema basado en mensajes. Es muy importante conocer qué mecanismo de sincronización se utilizará a la hora de manejar las primitivas *send* y *receive*, con el objetivo de determinar si las llamadas a dichas primitivas serán bloqueantes.

Sincronización

La comunicación de un mensaje entre dos procesos está asociada a un nivel de sincronización ya que, evidentemente, un proceso *A* no puede recibir un mensaje hasta que otro proceso *B* lo envía. En este contexto, se plantea la necesidad de asociar un comportamiento a un proceso cuando envía y recibe un mensaje.

Por ejemplo, cuando un proceso recibe un mensaje mediante una primitiva de **recepción**, existen dos posibilidades:

- Si el mensaje ya fue enviado por un proceso emisor, entonces el receptor puede obtener el mensaje y continuar su ejecución.

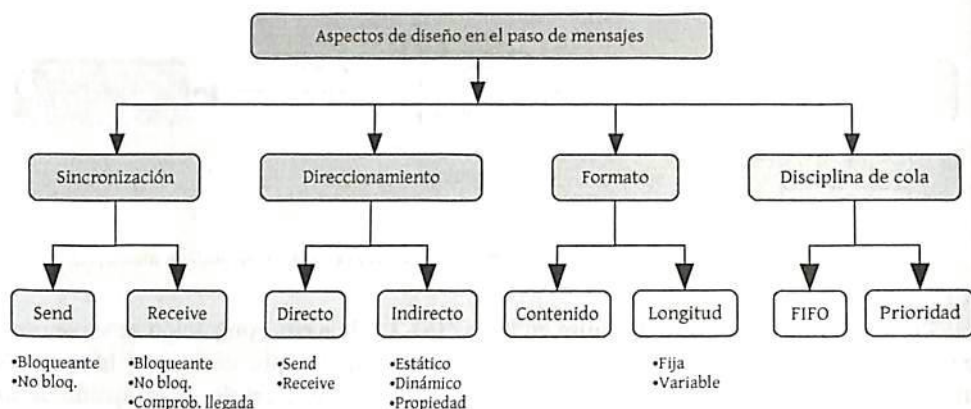


Figura 3.3: Aspectos básicos de diseño en sistemas de paso de mensajes para la comunicación y sincronización entre procesos.

- Si no existe un mensaje por obtener, entonces se plantean a su vez dos opciones:
 - Que el proceso receptor se bloquee hasta que reciba el mensaje.
 - Que el proceso receptor continúe su ejecución obviando, al menos temporalmente, el intento de recepción.

Del mismo modo, la operación de **envío** de mensajes se puede plantear de dos formas posibles:

- Que el proceso emisor se bloquee hasta que el receptor reciba el mensaje.
- Que el proceso emisor no se bloquee y se aísle, al menos temporalmente¹, del proceso de recepción del mensaje

En el ámbito de la programación concurrente, la **primitiva send** se suele utilizar con una naturaleza no bloqueante, con el objetivo de que un proceso pueda continuar su ejecución y, en consecuencia, pueda progresar en su trabajo. Sin embargo, este esquema puede fomentar prácticas no deseables, como por ejemplo que un proceso envíe mensajes de manera continuada sin que se produzca ningún tipo de penalización, asociada a su vez al bloqueo del proceso emisor. Desde el punto de vista del programador, este esquema hace que el propio programador sea el responsable de comprobar si un mensaje se recibió de manera adecuada, complicando así la lógica del programa.

En el caso de la **primitiva receive**, la versión bloqueante suele ser la elegida con el objetivo de sincronizar procesos. En otras palabras, un proceso que espera un mensaje suele necesitar la información contenida en el mismo para continuar con su ejecución. Un problema que surge con este esquema está derivado de la pérdida de un mensaje, ya que el proceso receptor se quedaría bloqueado. Dicho problema se puede abordar mediante una naturaleza no bloqueante.

¹ Considere el uso de un esquema basado en retrollamadas o notificaciones

Direccionamiento

Desde el punto de vista del envío y recepción de mensajes, resulta muy natural enviar y recibir mensajes de manera selectiva, es decir, especificando de manera explícita qué proceso enviará un mensaje y qué proceso lo va a recibir. Dicha información se podría especificar en las propias primitivas de envío y recepción de mensajes.

Recuperación por tipo

También es posible plantear un esquema basado en la recuperación de mensajes por tipo, es decir, una recuperación selectiva atendiendo al contenido del propio mensaje.

Este tipo de direccionamiento de mensajes recibe el nombre de **direccionamiento directo**, es decir, la primitiva *send* contiene un identificador de proceso vinculado al proceso emisor. En el caso de la primitiva *receive* se plantean dos posibilidades:

1. Designar de manera **explícita** al proceso emisor para que el proceso receptor sepa de quién recibe mensajes. Esta alternativa suele ser la más eficaz para procesos concurrentes cooperativos.
2. Hacer uso de un direccionamiento directo **implícito**, con el objetivo de modelar situaciones en las que no es posible especificar, de manera previa, el proceso de origen. Un posible ejemplo sería el proceso responsable del servidor de impresión.

El otro esquema general es el **direccionamiento indirecto**, es decir, mediante este esquema los mensajes no se envían directamente por un emisor a un receptor, sino que son enviados a una estructura de datos compartida, denominada buzón o cola de mensajes, que almacenan los mensajes de manera temporal. De este modo, el modelo de comunicación se basa en que un proceso envía un mensaje a la cola y otro proceso toma dicho mensaje de la cola.

La principal ventaja del direccionamiento indirecto respecto al directo es la flexibilidad, ya que existe un desacoplamiento entre los emisores y receptores de mensajes, posibilitando así diversos esquemas de comunicación y sincronización:

- uno a uno, para llevar a cabo una comunicación privada entre procesos.
- muchos a uno, para modelar situaciones cliente/servidor. En este caso, el buzón se suele conocer como *puerto*.
- uno a muchos, para modelar sistemas de *broadcast* o difusión de mensajes.
- muchos a muchos, para que múltiples clientes puedan acceder a un servicio concurrente proporcionado por múltiples servidores.

Finalmente, es importante considerar que la asociación de un proceso con respecto a un buzón puede ser estática o dinámica. Así mismo, el concepto de **propiedad** puede ser relevante para establecer relaciones entre el ciclo de vida de un proceso y el ciclo de vida de un buzón asociado al mismo.

Formato de mensaje

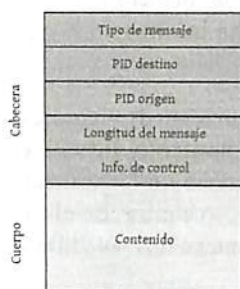


Figura 3.4: Formato típico de un mensaje, estructurado en cabecera y cuerpo. La cabecera contiene información básica para, por ejemplo, establecer un esquema dinámico asociado a la longitud del contenido del mensaje.

El formato de un mensaje está directamente asociado a la funcionalidad proporcionada por la biblioteca de gestión de mensajes y al ámbito de aplicación, entendiendo *ámbito* como su uso en una única máquina o en un sistema distribuido. En algunos sistemas de paso de mensajes, los mensajes tienen una **longitud fija** con el objetivo de no sobrecargar el procesamiento y almacenamiento de información. En otros, los mensajes pueden tener una **longitud variable** con el objetivo de incrementar la flexibilidad.

En la figura 3.4 se muestra el posible formato de un mensaje típico, distinguiendo entre la cabecera y el cuerpo del mensaje. Como se puede apreciar, en dicho mensaje van incluidos los identificadores únicos de los procesos emisor y receptor, conformando así un direccionamiento directo explícito.

Disciplina de cola

La disciplina de cola se refiere a la implementación subyacente que rige el comportamiento interno de la cola. Típicamente, las colas o buzones de mensajes se implementan haciendo uso de una cola **FIFO** (*First-In First-Out*), es decir, los mensajes primeros en salir son aquellos que llegaron en primer lugar.

Sin embargo, es posible utilizar una disciplina basada en el uso de un esquema basado en **prioridades**, asociando un valor numérico de importancia a los propios mensajes para establecer políticas de ordenación en base a los mismos.

3.1.3. El problema de la sección crítica



Figura 3.5: Visión gráfica del problema general de la sección crítica.

El problema de la sección crítica, introducido en la sección 1.2.2, se puede resolver fácilmente utilizando el mecanismo del paso de mensajes, tal y como muestra la figura 3.6.

Básicamente, es posible modelar una solución al problema de la sección crítica mediante la primitiva *receive* bloqueante. Así, los procesos comparten un único buzón, inicializado con un único mensaje, sobre el que envían y reciben mensajes. Un proceso que desee acceder a la sección crítica accederá al buzón mediante la primitiva *receive* para obtener un mensaje. En esta situación se pueden distinguir dos posibles casos:

1. Si el buzón está vacío, entonces el proceso se bloquea. Esta situación implica que, de manera previa, otro proceso accedió al buzón y recuperó el único mensaje del mismo. El primer proceso se quedará bloqueado hasta que pueda recuperar un mensaje del buzón.
2. Si el buzón tiene un mensaje, entonces el proceso lo recupera y puede acceder a la sección crítica.

En cualquier caso, el proceso ha de devolver el mensaje al buzón en su sección de salida para garantizar que el resto de procesos puedan acceder a su sección crítica. En esencia, el único mensaje del buzón se comporta como un *token* o testigo que gestiona el acceso exclusivo a la sección crítica. Es importante destacar que si existen varios procesos bloqueados a la espera de un mensaje en el buzón, entonces sólo uno de ellos se desbloqueará cuando otro proceso envíe un mensaje al buzón. No es posible conocer a priori qué proceso se desbloqueará aunque, típicamente, las colas de mensajes se implementan mediante colas FIFO (*first-in first-out*), por lo que aquellos procesos que se bloqueen en primer lugar serán los primeros en desbloquearse.



Figura 3.6: Mecanismo de exclusión mutua basado en el uso de paso de mensajes.

Desde un punto de vista semántico, el uso de un mecanismo basado en el paso de mensajes para modelar el problema de la sección crítica es idéntico al uso de un mecanismo basado en **semáforos**. Si se utiliza un semáforo para gestionar el acceso a la sección crítica, éste ha de inicializarse con un valor igual al número de procesos que pueden acceder a la sección crítica de manera simultánea. Típicamente, este valor será igual a uno para proporcionar un acceso exclusivo.

Si se utiliza el paso de mensajes, entonces el programador tendrá que *inicializar* el buzón con tantos mensajes como procesos puedan acceder a la sección crítica de manera simultánea (uno para un acceso exclusivo). La diferencia principal reside en que la inicialización es una operación propia del semáforo, mientras que en el paso de mensajes se ha de utilizar la primitiva *send* para rellenar el buzón y, de este modo, llevar a cabo la *inicialización* del mismo.

3.2. Implementación

En esta sección se discuten las **primitivas POSIX** más importantes relativas al paso de mensajes. Para ello, POSIX proporciona las denominadas colas de mensajes (*POSIX Message Queues*).

En el siguiente listado de código se muestra la primitiva *mq_open()*, utilizada para **abrir** una cola de mensajes existente o crear una nueva. Como se puede apreciar, dicha primitiva tiene dos versiones. La más completa incluye los permisos y una estructura para definir los atributos, además de establecer el nombre de la cola de mensajes y los *flags*.

Si la primitiva *mq_open()* abre o crea una cola de mensajes de manera satisfactoria, entonces devuelve el descriptor de cola asociada a la misma. Si no es así, devuelve *-1* y establece *errno* con el error generado.

Compilación

Utilice los *flags* adecuados en cada caso con el objetivo de detectar la mayor cantidad de errores en tiempo de compilación (e.g. escribir sobre una cola de mensajes de sólo lectura).

Al igual que ocurre con los semáforos nombrados, las colas de mensajes en POSIX se basan en la definición de un nombre específico para llevar a cabo la operación de apertura o creación de una cola.



Las colas de mensajes en POSIX han de empezar por el carácter '/', como por ejemplo '/BuzonMensajes1'.

Listado 3.1: Primitiva `mq_open` en POSIX

```
1 #include <fcntl.h>      /* Para constantes O_* */
2 #include <sys/stat.h>   /* Para los permisos */
3 #include <mqueue.h>
4
5 /* Devuelve el descriptor de la cola de mensajes */
6 /* o -1 si hubo error (ver errno) */
7 mqd_t mq_open (
8     const char *name,      /* Nombre de la cola */
9     int oflag              /* Flags (menos O_CREAT) */
10 );
11 mqd_t mq_open (
12     const char *name,      /* Nombre de la cola */
13     int oflag              /* Flags (incluyendo O_CREAT) */
14     mode_t perms,         /* Permisos */
15     struct mq_attr *attr  /* Atributos (o NULL) */
16 );
```

En el siguiente listado de código se muestra un ejemplo específico de creación de una cola de mensajes en POSIX. En las líneas [6-7] se definen los atributos básicos del buzón:

- `mq_maxmsg`, que define el máximo número de mensajes.
- `mq_msgsize`, que establece el tamaño máximo de mensaje.

Además de la posibilidad de establecer aspectos básicos relativos a una cola de mensajes a la hora de crearla, como el tamaño máximo del mensaje, es posible obtenerlos e incluso establecerlos de manera explícita, como se muestra en el listado de código 3.2.

Por otra parte, en la línea [10] se lleva a cabo la operación de creación de la cola de mensajes, identificada por el nombre `/prueba` y sobre la que el proceso que la crea podrá llevar a cabo operaciones de lectura (recepción) y escritura (envío). Note cómo la estructura relativa a los atributos previamente comentados se pasa por referencia. Finalmente, en las líneas [14-17] se lleva a cabo el control de errores básicos a la hora de abrir la cola de mensajes.

Listado 3.2: Obtención/modificación de atributos en una cola de mensajes en POSIX

```
1 #include <mqueue.h>
2
3 int mq_getattr (
4     mqd_t mqdes,          /* Descriptor de la cola */
5     struct mq_attr *attr  /* Atributos */
6 );
7
8 int mq_setattr (
9     mqd_t mqdes,          /* Descriptor de la cola */
10    struct mq_attr *newattr, /* Nuevos atributos */
11    struct mq_attr *oldattr  /* Antiguos atributos */
12 );
```

Las primitivas relativas al **cierre** y a la **eliminación** de una cola de mensajes son muy simples y se muestran en el siguiente listado de código. La primera de ellas necesita el descriptor de la cola, mientras que la segunda tiene como parámetro único el nombre asociado a la misma.

Listado 3.3: Ejemplo de creación de una cola de mensajes en POSIX

```
1 /* Descriptor de cola de mensajes */
2 mqd_t qHandler;
3 int rc;
4 struct mq_attr mqAttr;    /* Estructura de atributos */
5
6 mqAttr.mq_maxmsg = 10;    /* Máximo no de mensajes */
7 mqAttr.mq_msgsize = 1024; /* Tamaño máximo de mensaje */
8
9 /* Creación del buzón */
10 qHandler = mq_open('/prueba', O_RDWR | O_CREAT, S_IWUSR | S_IRUSR, &mqAttr);
11
12 /* Manejo de errores */
13 if (qHandler == -1) {
14     fprintf(stderr, "%s\n", strerror(errno));
15     exit(EXIT_FAILURE);
16 }
```

Desde el punto de vista de la programación concurrente, las primitivas más relevantes son las de **envío** y **recepción** de mensajes, ya que permiten no sólo sincronizar los procesos sino también compartir información entre los mismos. En el listado de código 3.4 se muestra la signatura de las primitivas POSIX *send* y *receive*.

Por una parte, la primitiva *send* permite el envío del mensaje definido por *msg_ptr*, y cuyo tamaño se especifica en *msg_len*, a la cola asociada al descriptor especificado por *mqdes*. Es posible asociar una prioridad a los mensajes que se envían a una cola mediante el parámetro *msg_prio*.

Listado 3.4: Primitivas de cierre y eliminación de una cola de mensajes en POSIX

```

1 #include <mqueue.h>
2
3 int mq_close (
4     mqd_t mqdes      /* Descriptor de la cola de mensajes */
5 );
6
7 int mq_unlink (
8     const char *name  /* Nombre de la cola */
9 );

```

Por otra parte, la primitiva *receive* permite recibir mensajes de una cola. Note cómo los parámetros son prácticamente los mismos que los de la primitiva *send*. Sin embargo, ahora *msg_ptr* representa el buffer que almacenará el mensaje a recibir y *msg_len* es el tamaño de dicho buffer. Además, la primitiva de recepción implica obtener la prioridad asociada al mensaje a recibir, en lugar de especificarla a la hora de enviarlo.

Listado 3.5: Primitivas de envío y recepción en una cola de mensajes en POSIX

```

1 #include <mqueue.h>
2
3 int mq_send (
4     mqd_t mqdes,      /* Descriptor de la cola */
5     const char *msg_ptr, /* Mensaje */
6     size_t msg_len,    /* Tamaño del mensaje */
7     unsigned msg_prio  /* Prioridad */
8 );
9
10 ssize_t mq_receive (
11     mqd_t mqdes,      /* Descriptor de la cola */
12     char *msg_ptr,     /* Buffer para el mensaje */
13     size_t msg_len,    /* Tamaño del buffer */
14     unsigned *msg_prio /* Prioridad o NULL */
15 );

```

El listado de código 3.6 muestra un ejemplo muy representativo del uso del paso de mensajes en POSIX, ya que refleja la **recepción bloqueante**, una de las operaciones más representativas de este mecanismo.

En las líneas [6-7] se refleja el uso de la primitiva *receive*, utilizando *buffer* como variable receptora del contenido del mensaje a recibir y *prioridad* para obtener el valor de la misma. Recuerde que la semántica de las operaciones de una cola, como por ejemplo que la recepción sea no bloqueante, se definen a la hora de su creación, tal y como se mostró en uno de los listados anteriores, mediante la estructura *mq_attr*. Sin embargo, es posible modificar y obtener los valores de configuración asociados a una cola mediante las primitivas *mq_setattr* y *mq_getattr*, respectivamente.



El buffer de recepción de mensajes ha de tener un tamaño mayor o igual a *mq_msgsize*, mientras que el buffer de envío de un mensaje ha de tener un tamaño menor o igual a *mq_msgsize*.

Listado 3.6: Ejemplo de recepción bloqueante en una cola de mensajes en POSIX

```
1 mqd_t qHandler;
2 int rc;
3 unsigned int prioridad;
4 char buffer[2048];
5
6 rc = mq_receive(qHandler, buffer, sizeof(buffer), &prioridad);
7
8 if (rc == -1) {
9     fprintf(stderr, "%s\n", strerror(errno));
10 }
11 else {
12     printf ("Recibiendo mensaje: %s\n", buffer);
13 }
```

En el listado 3.6 también se expone cómo obtener información relativa a posibles errores mediante *errno*. En este contexto, el fichero de cabecera *errno.h*, incluido en la biblioteca estándar del lenguaje C, define una serie de macros para informar sobre condiciones de error, a través de una serie de códigos de error, almacenados en un espacio estático denominado *errno*.

De este modo, algunas primitivas modifican el valor de este espacio cuando detectan determinados errores. En el ejemplo de recepción de mensajes, si *receive* devuelve un código de error -1, entonces el programador puede obtener más información relativa al error que se produjo, como se aprecia en las líneas 9-10.

El **envío de mensajes**, en el ámbito de la programación concurrente, suele mantener una semántica no bloqueante, es decir, cuando un proceso envía un mensaje su flujo de ejecución continúa inmediatamente después de enviar el mensaje. Sin embargo, si la cola POSIX ya estaba llena de mensajes, entonces hay que distinguir entre dos casos:

En POSIX...

En el estándar POSIX, las primitivas *mq_timedsend* y *mq_timedreceive* permiten asociar *timeouts* para especificar el tiempo máximo de bloqueo al utilizar las primitivas *send* y *receive*, respectivamente.

1. Por defecto, un proceso se bloqueará al enviar un mensaje a una cola llena, hasta que haya espacio disponible para encolarlo.
2. Si el flag *O_NONBLOCK* está activo en la descripción de la cola de mensajes, entonces la llamada a *send* fallará y devolverá un código de error.

El listado de código 3.7 muestra un **ejemplo de envío** de mensajes a través de la primitiva *send*. Como se ha comentado anteriormente, esta primitiva de envío de mensajes se puede comportar de una manera no bloqueante cuando la cola está llena de mensajes (ha alcanzado el número máximo definido por *mq_maxmsg*).

Listado 3.7: Ejemplo de envío en una cola de mensajes en POSIX

```

1 mqd_t qHandler;
2 int rc;
3 unsigned int prioridad;
4 char buffer[512];
5
6 sprintf (buffer, "[ Saludos de %d ]", getpid());
7 mq_send(qHandler, buffer, sizeof(buffer), 1);

```

Llegados a este punto, el lector se podría preguntar cómo es posible incluir tipos o **estructuras de datos específicas** de un determinado dominio con el objetivo de intercambiar información entre varios procesos mediante el uso del paso de mensajes. Esta duda se podría plantear debido a que tanto la primitiva de envío como la primitiva de recepción de mensajes manejan cadenas de caracteres (punteros a *char*) para gestionar el contenido del mensaje.

En realidad, este enfoque, extendido en diversos ámbitos del estandar POSIX, permite manejar cualquier tipo de datos de una manera muy sencilla, ya que sólo es necesario aplicar los moldes necesarios para enviar cualquier tipo de información. En otras palabras, el tipo de datos *char ** se suele utilizar como buffer genérico, posibilitando el uso de cualquier tipo de datos definido por el usuario.



Muchos APIs utilizan *char ** como buffer genérico. Simplemente, considere un puntero a buffer, junto con su tamaño, a la hora de manejar estructuras de datos como contenido de mensaje en las primitivas *send* y *receive*.

El listado de código 3.8 muestra un ejemplo representativo de esta problemática. Como se puede apreciar, en dicho ejemplo el buffer utilizado es una estructura del tipo *TData*, definida por el usuario. Note cómo al usar las primitivas *send* y *receive* se hace uso de los moldes correspondientes para cumplir con la especificación de dichas primitivas.

Listado 3.8: Uso de tipos de datos específicos para manejar mensajes

```

1 typedef struct {
2     int valor;
3 } TData;
4
5 TData buffer;
6
7 /* Atributos del buzón */
8 mqAttr.mq_maxmsg = 10;
9 mqAttr.mq_msgsize = sizeof(TData);
10
11 /* Recepción */
12 mq_receive(qHandler, (char *)&buffer, sizeof(buffer), &prioridad);
13 /* Envío */
14 mq_send(qHandler, (const char *)&buffer, sizeof(buffer), 1);

```

Finalmente, el estándar POSIX también define la primitiva `mq_notify` en el contexto de las colas de mensajes con el objetivo de **notificar a un proceso** o hilo, mediante una señal, cuando llega un mensaje.

Básicamente, esta primitiva permite que cuando un proceso o un hilo se registre, éste reciba una señal cuando llegue un mensaje a una cola vacía especificada por `mqdes`, a menos que uno o más procesos o hilos ya estén bloqueados mediante una llamada a `mq_receive`. En esta situación, una de estas llamadas a dicha primitiva retornará en su lugar.

Listado 3.9: Primitiva `mq_notify` en POSIX

```
1 #include <mqqueue.h>
2
3 int mq_notify (
4     /* Descriptor de la cola */
5     mqd_t mqdes,
6     /* Estructura de notificación
7      * Contiene funciones de retrolamada
8      * para notificar la llegada de un nuevo mensaje */
9     const struct sigevent *sevp
10 );
```

3.2.1. El problema del bloqueo

El uso de distintas colas de mensajes, asociado al manejo de distintos tipos de tareas o de distintos tipos de mensajes, por parte de un proceso que maneje una semántica de recepción bloqueante puede dar lugar al denominado *problema del bloqueo*.

Suponga que un proceso ha sido diseñado para atender peticiones de varios buzones, nombrados como A y B. En este contexto, el proceso se podría bloquear a la espera de un mensaje en el buzón A. Mientras el proceso se encuentra bloqueado, otro mensaje podría llegar al buzón B. Sin embargo, y debido a que el proceso se encuentra bloqueado en el buzón A, el mensaje almacenado en B no podría ser atendido por el proceso, al menos hasta que se desbloqueara debido a la nueva existencia de un mensaje en A.



¿Cómo es posible atender distintas peticiones (tipos de tareas) de manera eficiente por un conjunto de procesos?

Esta problemática es especialmente relevante en las colas de mensajes POSIX, donde no es posible llevar a cabo una recuperación selectiva por tipo de mensaje².

Existen diversas **soluciones** al problema del bloqueo. Algunas de ellas son específicas del interfaz de paso de mensajes utilizado, mientras que otras, como la que se discutirá a continuación, son generales y se podrían utilizar con cualquier sistema de paso de mensajes.

²En System V IPC, es posible recuperar mensajes de un buzón indicando un tipo.

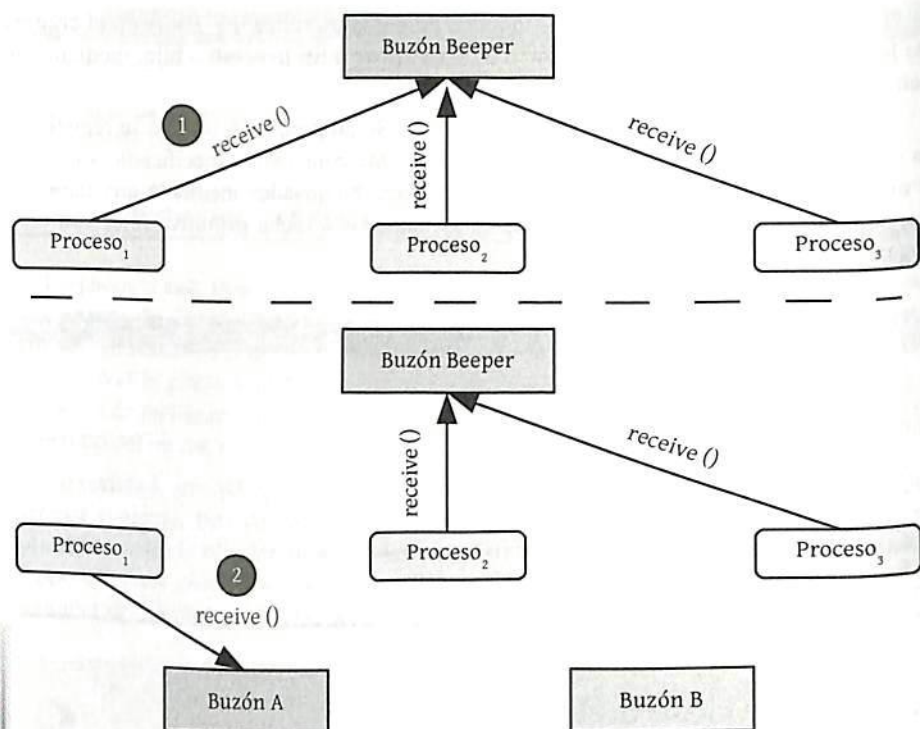


Figura 3.7: Mecanismo de notificación basado en un buzón o cola tipo *beeper* para gestionar el problema del bloqueo.

Dentro del contexto de las situaciones específicas de un sistema de paso de mensajes, es posible utilizar un esquema basado en **recuperación selectiva**, es decir, un esquema que permite que los procesos obtengan mensajes de un buzón en base a un determinado criterio. Por ejemplo, este criterio podría definirse en base al tipo de mensaje, codificado mediante un valor numérico. Otro criterio más exclusivo podría ser el propio identificador del proceso, planteando así un esquema estrechamente relacionado con una comunicación directa.

No obstante, el problema de la recuperación selectiva sólo solventa el problema del bloqueo parcialmente ya que no es posible conocer a priori si un proceso se bloqueará ante la recepción (con semántica bloqueante) de un mensaje asociado a un determinado tipo.

Otra posible opción específica, en este caso incluida en las colas de mensajes de POSIX, consiste en hacer uso de algún tipo de esquema de notificación que sirva para que un proceso sepa cuándo se ha recibido un mensaje en un buzón específico. En el caso particular de POSIX, la operación *mq_notify* permite, previo registro, que un proceso o hilo reciba una señal cuando un mensaje llegue a un determinado buzón.

Desde el punto de vista de las soluciones generales, en la sección 5.18 de [9] se discute una solución al problema del bloqueo denominada **Unified Event Manager** (gestor de eventos unificado). Básicamente, este enfoque propone una biblioteca de funciones que cualquier aplicación puede usar. Así, una aplicación puede registrar un evento, causando que la biblioteca cree un hilo que se bloquea. La aplicación está organizada alrededor de una cola con un único evento. Cuando un evento llega, la aplicación lo desencola, lo procesa y vuelve al estado de espera.

El enfoque general que se plantea en esta sección se denomina **esquema beeper** (ver figura 3.7) y consiste en utilizar un buzón específico que los procesos consultarán para poder recuperar información relevante sobre la próxima tarea a acometer.



El esquema *beeper* planteado en esta sección se asemeja al concepto de localizador o buscador de personas, de manera que el localizador notifica el número de teléfono o el identificador de la persona que está buscando para que la primera se pueda poner en contacto.

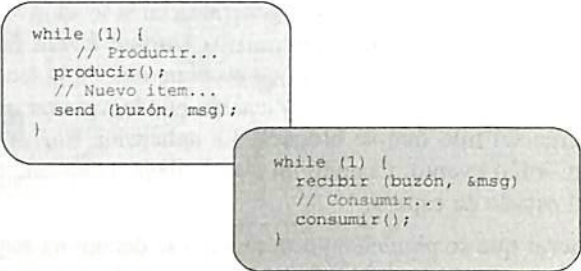
Por ejemplo, un proceso que obtenga un mensaje del buzón *beeper* podrá utilizar la información contenido en el mismo para acceder a otro buzón particular. De este modo, los procesos tienen más información para acometer la siguiente tarea o procesar el siguiente mensaje.

En base a este esquema general, los procesos estarán bloqueados por el buzón *beeper* a la hora de recibir un mensaje. Cuando un proceso lo reciba y, por lo tanto, se desbloquee, entonces podrá utilizar el contenido del mensaje para, por ejemplo, acceder a otro buzón que le permita completar una tarea. En realidad, el uso de este esquema basado en dos niveles evita que un proceso se bloquee en un buzón mientras haya tareas por atender en otro.

Así, los procesos estarán típicamente bloqueados mediante la primitiva *receive* en el buzón *beeper*. Cuando un cliente requiera que uno de esos procesos atienda una petición, entonces enviará un mensaje al buzón *beeper* indicando en el contenido el tipo de tarea o el nuevo buzón sobre el que obtener información. Uno de los procesos bloqueados se desbloqueará, consultará el contenido del mensaje y, posteriormente, realizará una tarea o accederá al buzón indicado en el contenido del mensaje enviado al *beeper*.

3.3. Problemas clásicos de sincronización

En esta sección se plantean varios problemas clásicos de sincronización, algunos de los cuales ya se han planteado en la sección 2.3, y se discuten cómo podrían solucionarse utilizando el mecanismo del paso de mensajes.



```
while (1) {  
    // Producir...  
    producir();  
    // Nuevo item...  
    send (buzón, msg);  
}
```

```
while (1) {  
    recibir (buzón, &msg)  
    // Consumir...  
    consumir();  
}
```

Figura 3.8: Solución al problema del buffer limitado mediante una cola de mensajes.

3.3.1. El buffer limitado

El problema del buffer limitado o *problema del productor/consumidor*, introducido en la sección 1.2.1, se puede resolver de una forma muy sencilla utilizando un único buzón o cola de mensajes. Recuerde que en la sección 2.3.1 se planteó una solución a este problema utilizando dos semáforos contadores para controlar, respectivamente, el número de huecos llenos y el número de huecos vacíos en el buffer.

Sin embargo, la solución mediante paso de mensajes es más simple, ya que sólo es necesario utilizar un único buzón para controlar la problemática subyacente. Básicamente, tanto los productores como los consumidores comparten un buzón cuya capacidad está determinada por el número de huecos del buffer.

Inicialmente, el buzón se *rellena* o *inicializa* con tantos mensajes como huecos ocupados haya originalmente, es decir, 0. Tanto los productores como los consumidores utilizan una semántica bloqueante, de manera que

- Si un productor intenta insertar un elemento con el buffer lleno, es decir, si intenta enviar un mensaje al buzón lleno, entonces se quedará bloqueado a la espera de un nuevo hueco (a la espera de que un consumidor obtenga un item del buffer y, en consecuencia, un mensaje de la cola).
- Si un consumidor intenta obtener un elemento con el buffer vacío, entonces se bloqueará a la espera de que algún consumidor inserte un nuevo elemento. Esta situación se modela mediante la primitiva de recepción bloqueante.

Básicamente, la cola de mensajes permite modelar el comportamiento de los procesos productor y consumidor mediante las primitivas básicas de envío y recepción bloqueantes.

3.3.2. Los filósofos comensales

El problema de los filósofos comensales, discutido en la sección 2.3.3, también se puede resolver de una manera sencilla utilizando el paso de mensajes y garantizando incluso que no haya un interbloqueo.

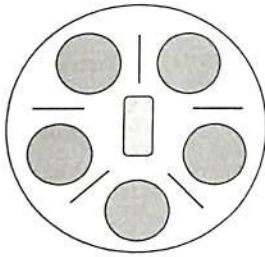


Figura 3.9: Esquema gráfico del problema original de los filósofos comensales (*dining philosophers*).

Al igual que ocurría con la solución basada en el uso de semáforos, la solución discutida en esta sección maneja un **array de palillos** para gestionar el acceso exclusivo a los mismos. Sin embargo, ahora cada palillo está representado por una cola de mensajes con un único mensaje que actúa como *token* o testigo entre los filósofos. En otras palabras, para que un filósofo pueda obtener un palillo, tendrá que recibir el mensaje de la cola correspondiente. Si la cola está vacía, entonces se quedará bloqueado hasta que otro filósofo envíe el mensaje, es decir, libere el palillo.

En el listado 3.10 se muestra el **pseudocódigo** de una posible solución a este problema utilizando paso de mensajes. Note cómo se utiliza un array, denominado *palillo*, donde cada elemento de dicho array representa un buzón de mensajes con un único mensaje.

Listado 3.10: Solución a los filósofos (sin interbloqueo) con colas de mensajes

```

1 while (1) {
2     /* Pensar... */
3     receive(mesa, &m);
4     receive(palillo[i], &m);
5     receive(palillo[i + 1], &m);
6     /* Comer... */
7     send(palillo[i], m);
8     send(palillo[i + 1], m);
9     send(mesa, m);
10 }
```

Por otra parte, la solución planteada también hace uso de un buzón, denominado *mesa*, que tiene una capacidad de cuatro mensajes. Inicialmente, dicho buzón se rellena con cuatro mensajes que representan el número máximo de filósofos que pueden coger palillos cuando se encuentren en el estado *hambriento*. El objetivo es evitar una situación de interbloqueo que se podría producir si, por ejemplo, todos los filósofos cogen a la vez el palillo que tienen a su izquierda.

Al manejar una semántica bloqueante a la hora de recibir mensajes (intentar adquirir un palillo), la solución planteada garantiza el acceso exclusivo a cada uno de los palillos por parte de un único filósofo. La acción de dejar un palillo sobre la mesa está representada por la primitiva *send*, posibilitando así que otro filósofo, probablemente bloqueado por *receive*, pueda adquirir el palillo para comer.

En el anexo B se muestra una **implementación** al problema de los filósofos comensales utilizando las colas de mensajes POSIX. En dicha implementación, al igual que en la solución planteada en la presente sección, se garantiza que no exista un interbloqueo entre los filósofos.

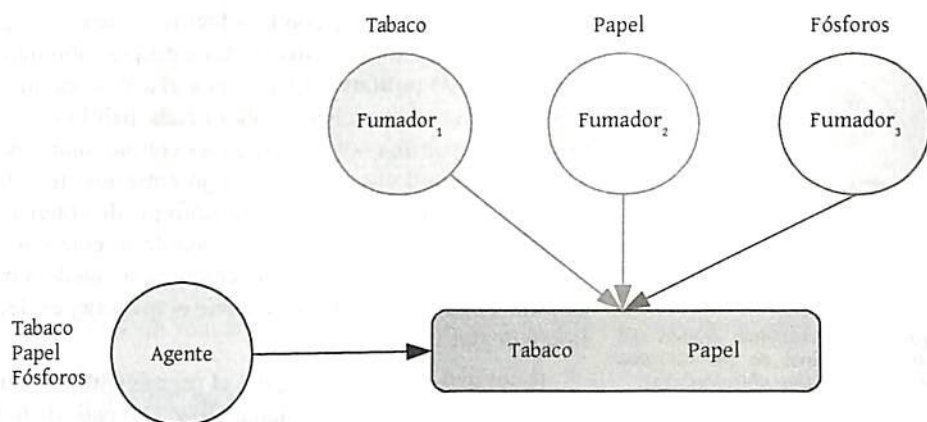


Figura 3.10: Esquema gráfico del problema de los fumadores de cigarrillos.

3.3.3. Los fumadores de cigarrillos

El problema de los fumadores de cigarrillos es otro problema clásico de sincronización. Básicamente, es un problema de **coordinación** entre un agente y tres fumadores en el que intervienen tres ingredientes: i) papel, ii) tabaco y iii) fósforos. Por una parte, el agente dispone de una cantidad ilimitada respecto a los tres ingredientes. Por otra parte, cada fumador dispone de un único elemento, también en cantidad ilimitada, es decir, un fumador dispone de papel, otro de tabaco y otro de fósforos.

Cada cierto tiempo, el agente coloca sobre una mesa, de manera aleatoria, dos de los tres ingredientes necesarios para liarse un cigarrillo (ver figura 3.10). El fumador que tiene el ingrediente restante coge los otros dos de la mesa, se lia el cigarrillo y se lo notifica al agente. Este ciclo se repite indefinidamente.

La problemática principal reside en coordinar adecuadamente al agente y a los fumadores, considerando las siguientes **acciones**:

1. El agente pone dos elementos sobre la mesa.
2. El agente notifica al fumador que tiene el tercer elemento que ya puede liarse un cigarrillo (se simula el acceso del fumador a la mesa).
3. El agente espera a que el fumador le notifique que ya ha terminado de liarse el cigarrillo.
4. El fumador notifica que ya ha terminado de liarse el cigarrillo.
5. El agente vuelve a poner dos nuevos elementos sobre la mesa, repitiéndose así el ciclo.

Proceso agente

```

while (1) {
  /* Genera ingredientes */
  restante = poner_ingredientes();
  /* Notifica al fumador */
  send(buzones[restante], msg);
  /* Espera notificación fumador */
  receive(buzon_agente, &msg);
}

```

Proceso fumador

```

while (1) {
  /* Espera ingredientes */
  receive(mi_buzon, &msg);
  // Liando...
  liar_cigarrillo();
  /* Notifica al agente */
  send(buzon_agente, msg);
}

```

Figura 3.11: Solución en pseudocódigo al problema de los fumadores de cigarrillos usando paso de mensajes.

Este problema se puede resolver utilizando dos posibles esquemas:

- Recuperación **selectiva** utilizando un único buzón (ver figura 3.12). En este caso, es necesario integrar en el contenido del mensaje de algún modo el tipo asociado al mismo con el objetivo de que lo recupere el proceso adecuado. Este tipo podría ser A, T, P o F en función del destinatario del mensaje.
- Recuperación **no selectiva** utilizando múltiples buzones (ver figura 3.13). En este caso, es necesario emplear tres buzones para que el agente pueda comunicarse con los tres tipos de agentes y otro buzón para que los fumadores puedan indicarle al agente que ya terminaron de liarse un cigarrillo.

Este último caso de recuperación no selectiva está estrechamente ligado al planteamiento de las colas de mensajes POSIX, donde no existe un soporte explícito para llevar a cabo una recuperación selectiva. En la figura 3.11 se muestra el pseudocódigo de una posible solución atendiendo a este segundo esquema.

Como se puede apreciar en la misma, el agente notifica al fumador que completa los tres ingredientes necesarios para fumar a través de un buzón específico, asociado a cada tipo de fumador. Posteriormente, el agente se queda bloqueado (mediante *receive* sobre *buzon_agente*) hasta que el fumador notifique que ya ha terminado de liarse el cigarrillo. En esencia, la sincronización entre el agente y el fumador se realiza utilizando el **patrón rendezvous**.

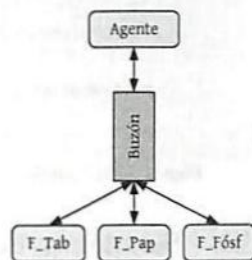


Figura 3.12: Solución al problema de los fumadores de cigarrillos con un único buzón de recuperación selectiva.

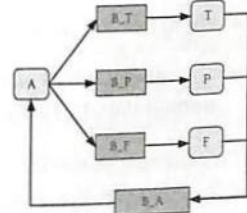


Figura 3.13: Solución al problema de los fumadores de cigarrillos con múltiples buzones sin recuperación selectiva (A=Agente, T=Fumador con tabaco, P=Fumador con papel, F=Fumador con fósforos).

Entero	1	2	...	25	26	27	28	...	51	52	53	54	55	56	57
Traducción	a	b	...	y	z	A	B	...	Y	Z	.	,	!	?	_
Código ASCII	97	98	...	121	122	65	66	...	89	90	46	44	33	63	95

Figura 3.14: Esquema con las correspondencias de traducción. Por ejemplo, el carácter codificado con el valor 1 se corresponde con el código ASCII 97, que representa el carácter 'a'.

Cadena de entrada	34	15	12	1	55
Cadena descodificada	72	111	108	97	33
Representación textual	'H'	'o'	'l'	'a'	'!'

Figura 3.15: Ejemplo concreto de traducción mediante el sistema de codificación discutido.

3.3.4. Simulación de un sistema de codificación

En esta sección se propone un problema que no se encuadra dentro de los problemas clásicos de sincronización pero que sirve para afianzar el uso del sistema de paso de mensajes para sincronizar procesos y comunicarlos.

Básicamente, se pide construir un sistema que simule a un sencillo sistema de codificación de caracteres por parte de una serie de **procesos de traducción**. En este contexto, existirá un **proceso cliente** encargado de manejar la siguiente información:

- Cadena codificada a descodificar.
- Número total de procesos de traducción.
- Tamaño máximo de subcadena a descodificar, es decir, tamaño de la unidad de trabajo máximo a enviar a los procesos de codificación.

La cadena a descodificar estará formada por una secuencia de números enteros, entre 1 y 57, separados por puntos, que los procesos de traducción tendrán que traducir utilizando el sistema mostrado en la figura 3.14. El tamaño de cada una de las subcadenas de traducción vendrá determinado por el tercer elemento mencionado en el listado anterior, es decir, el tamaño máximo de subcadena a descodificar.

Por otra parte, el sistema contará con un **único proceso de puntuación**, encargado de traducir los símbolos de puntuación correspondientes a los valores enteros comprendidos en el rango 53-57, ambos inclusive. Así, cuando un proceso de traducción encuentre uno de estos valores, entonces tendrá que enviar un mensaje al único proceso de puntuación para que éste lo descodifique y, posteriormente, se lo envíe al cliente mediante un mensaje.

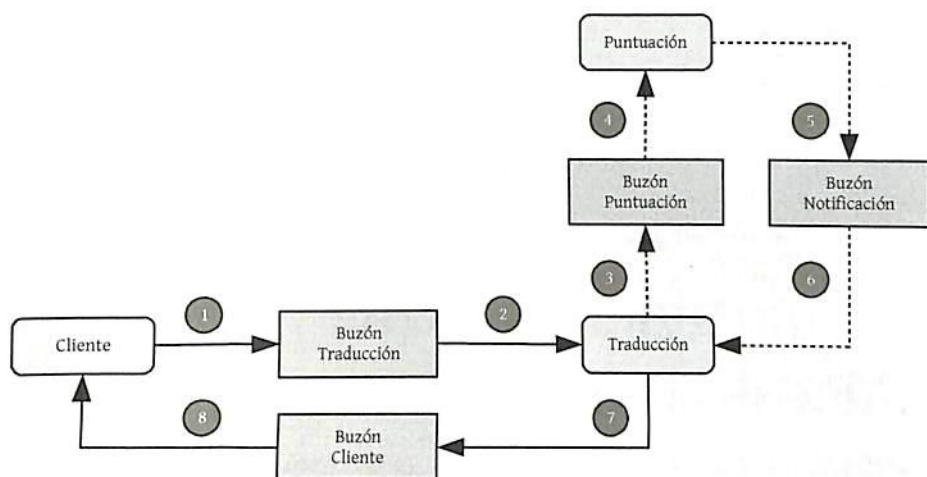


Figura 3.16: Esquema gráfico con la solución planteada, utilizando un sistema de paso de mensajes, para sincronizar y comunicar a los procesos involucrados en la simulación planteada. Las líneas punteadas indican pasos opcionales (traducción de símbolos de puntuación).

Los procesos de traducción atenderán peticiones hasta que el cliente haya enviado todas las subcadenas asociadas a la cadena de traducción original. En la siguiente figura se muestra un ejemplo concreto con una cadena de entrada y el resultado de su descodificación.

La figura 3.16 muestra de manera gráfica el diseño, utilizando colas de mensajes POSIX (sin recuperación selectiva), planteado para solucionar el problema propuesto en esta sección. Básicamente, se utilizan cuatro colas de mensajes en la solución planteada:

- **Buzón de traducción**, utilizado por el proceso cliente para enviar las subcadenas a descodificar. Los procesos de traducción estarán bloqueados mediante una primitiva de recepción sobre dicho buzón a la espera de trabajo.
- **Buzón de puntuación**, utilizado por los procesos de traducción para enviar mensajes que contengan símbolos de puntuación al proceso de puntuación.
- **Buzón de notificación**, utilizado por el proceso de puntuación para enviar la información descodificada al proceso de traducción que solicitó la tarea.
- **Buzón de cliente**, utilizado por los procesos de traducción para enviar mensajes con subcadenas descodificadas.

En la solución planteada se ha aplicado el patrón *rendezvous* en dos ocasiones para sincronizar a los distintos procesos involucrados. En concreto,

- El proceso cliente envía una serie de órdenes a los procesos de traducción, quedando posteriormente a la espera hasta que haya obtenido todos los resultados parciales (mismo número de órdenes enviadas).
- El proceso de traducción envía un mensaje al proceso de puntuación cuando encuentra un símbolo de traducción, quedando a la espera, mediante la primitiva *receive*, de que este último envíe un mensaje con la traducción solicitada.

Proceso cliente

```

obtener_datos(&datos);
crear_buzones();
lanzar_procesos();

calcular_num_subv(datos, &num_sub);

for i in range (0, num_subv) {
    // Generar orden.
    generar_orden(&orden, datos);
    // Solicita traducción.
    send(buzón_traducción, orden);
}

for i in range (0, num_subv) {
    // Espera traducción.
    receive(buzon_cliente, &msg);
    actualizar(datos, msg);
}

mostrar_resultado (datos);

```

Proceso traducción

```

while (1) {
    // Espera tarea.
    receive(buzón_traducción, &orden);
    // Traducción...
    if (es_simbolo(orden.datos)) {
        send(buzón_puntuación, orden);
        receive(buzón_notificación, &msg);
    }
    else
        traducir(&orden, &msg);
    // Notifica traducción.
    send(buzon_cliente, msg);
}

```

Proceso puntuación

```

while (1) {
    receive(buzón_puntuación, &orden);
    traducir(&orden, &msg);
    send(buzon_notificación, msg);
}

```

Figura 3.17: Solución en pseudocódigo que muestra la funcionalidad de cada uno de los procesos del sistema de codificación.

La figura 3.17 muestra una posible solución en pseudocódigo considerando un sistema de paso de mensajes con recuperación no selectiva.

Como se puede apreciar, el **proceso cliente** es el responsable de obtener el número de subvectores asociados a la tarea de traducción para, posteriormente, enviar tantos mensajes a los procesos de traducción como tareas haya. Una vez enviados los mensajes al buzón de traducción, el proceso cliente queda a la espera, mediante la primitiva de recepción bloqueante, de los distintos resultados parciales. Finalmente, mostrará el resultado descodificado.

Por otra parte, el **proceso de traducción** permanece en un bucle infinito a la espera de algún trabajo, utilizando para ello la primitiva de recepción bloqueante sobre el buzón de traducción (utilizado por el cliente para enviar trabajos). Si un trabajo contiene un símbolo de traducción, entonces el proceso de traducción solicita ayuda al proceso de puntuación mediante un *rendezvous*, utilizando el buzón de puntuación. Si no es así, entonces traduce el subvector directamente. En cualquier caso, el proceso de traducción envía los datos descodificados al proceso cliente.

Finalmente, el **proceso de puntuación** también permanece en un bucle infinito, bloqueado en el buzón de puntuación, a la espera de alguna tarea. Si la recibe, entonces la procesa y envía el resultado, mediante el buzón de notificación, al proceso de traducción que solicitó dicha tarea.

En este problema se puede apreciar la potencia de los sistemas de paso de mensajes, debido a que se pueden utilizar no sólo para sincronizar procesos sino también para compartir información.